



Career
Credentials

C++ LOGIC BUILDING Workbook

Basic to Advanced – Topic by Topic



I/O

Conditions

Loops

Strings

Arrays

OOP

STL

RTL

Files

How to Use This Booklet

This workbook is structured so that every chapter follows the exact same pattern:

- A 1–1.5 page Cheat Sheet recapping the theory — syntax, operators, and key concepts.
- A graded Problem Set divided into Basic, Intermediate, and Advanced questions.
- Space to think: always attempt each problem on paper before typing it.

TIP

Treat each chapter as a sprint. Read the cheat sheet once, close it, and solve the first 3 problems from memory. Return only if you are truly stuck.

Level	What it means
Basic	Direct application of one concept. Usually 5–10 lines of code.
Intermediate	Combines two or more ideas. Requires some planning before coding.
Advanced	Real-world scenario or algorithmic twist. Design before you code.

C H A P T E R 1

Input / Output Operators

cin, cout, cerr, formatting & type casting

Cheat Sheet — I/O Operators

Standard Streams

Object	Purpose & Header
<code>cin</code>	Standard input (keyboard). Header: <code><iostream></code>
<code>cout</code>	Standard output (screen). Header: <code><iostream></code>
<code>cerr</code>	Standard error output (unbuffered). Header: <code><iostream></code>
<code>clog</code>	Standard log output (buffered). Header: <code><iostream></code>

cin & cout Operators

Operator / Syntax	Example	Description
<code>>></code>	<code>cin >> x >> y;</code>	Extraction: reads value from input
<code><<</code>	<code>cout << x << endl;</code>	Insertion: sends value to output
<code>endl</code>	<code>cout << x << endl;</code>	Flush buffer + newline
<code>"\n"</code>	<code>cout << "hi\n";</code>	Newline only (faster than endl)
<code>cin.ignore()</code>	<code>cin.ignore(100, '\n');</code>	Discard characters from buffer
<code>getline()</code>	<code>getline(cin, str);</code>	Read full line including spaces
<code>cin.get()</code>	<code>char c = cin.get();</code>	Read single character

Format Manipulators (#include <iomanip>)

Operator / Syntax	Example	Description
<code>setw(n)</code>	<code>setw(10) << x</code>	Set field width for next output
<code>setfill(c)</code>	<code>setfill('0') << x</code>	Set fill character
<code>setprecision(n)</code>	<code>setprecision(2) << x</code>	Decimal digits (use with fixed)
<code>fixed</code>	<code>fixed << 3.14</code>	Fixed-point notation
<code>scientific</code>	<code>scientific << 3.14</code>	Scientific notation
<code>left / right</code>	<code>left << setw(10) << x</code>	Alignment within field width
<code>boolalpha</code>	<code>boolalpha << true</code>	Print true/false not 1/0

hex/oct/dec

hex << 255

Change number base

Quick Code Pattern

```
// Complete I/O Example
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int age;  string name;  double gpa;

    cout << "Enter name: ";
    getline(cin, name);
    cout << "Enter age and GPA: ";
    cin >> age >> gpa;

    cout << fixed << setprecision(2);
    cout << left << setw(12) << name
         << setw(5)  << age
         << setw(6)  << gpa << endl;
    return 0;
}
```

Problem Set — I/O Operators

#	Level	I/O Operator Problems
1	Basic	Write a program to read your name, age, and city, then print them in a single formatted line.
2	Basic	Read two integers and print their sum, difference, product, and quotient on separate lines.
3	Basic	Read a floating-point number and print it with exactly 3 decimal places using setprecision.
4	Basic	Print the numbers 1 to 5 in a right-aligned column of width 8 using setw.
5	Basic	Read a character using cin.get() and print its ASCII value.
6	Basic	Use getline() to read a full sentence (with spaces) and print its length.
7	Basic	Print 255 in decimal, hexadecimal, and octal format using manipulators.
8	Intermediate	Read the names and marks of 5 students and print a neatly formatted table with headers (Name, Marks, Grade).
9	Intermediate	Build a simple bill generator: read item name, quantity, and unit price; print a formatted receipt with total.
10	Intermediate	Read a 3x3 matrix of integers and print it as a grid using setw for alignment.

11	Intermediate	Read a boolean value as 'true' or 'false' using boolalpha and print the opposite.
12	Intermediate	Create a temperature conversion tool: read Celsius and print Fahrenheit and Kelvin with 4 decimal places.
13	Advanced	Simulate a bank statement: read 5 transactions (debit/credit amounts) and print a formatted statement with running balance.
14	Advanced	Build a progress bar: read a percentage (0-100) and print a visual bar like [=====>] 80% using setfill and setw.
15	Advanced	Read a student's data (name, rollno, 5 subject marks) and print a fully formatted mark sheet with total, percentage, and grade.

C H A P T E R 2

Conditional Statements

if, if-else, if-else-if, switch, ternary



Cheat Sheet — Conditional Statements

The if Statement — execute block only when condition is true

```
// if syntax
if (condition) {
    // runs when condition == true
}

// Example
if (marks >= 90) cout << "Grade A";
```

if-else — two-way decision

```
// if-else
if (num % 2 == 0) cout << "Even";
else cout << "Odd";
```

if-else-if Ladder — multiple exclusive conditions

```
// if-else-if
if (marks >= 90) cout << "A";
else if (marks >= 75) cout << "B";
else if (marks >= 60) cout << "C";
else if (marks >= 40) cout << "D";
else cout << "Fail";
```

switch — compare one variable against constant values

```
// switch
switch (day) { // expression must be int or char
    case 1: cout << "Monday"; break;
    case 2: cout << "Tuesday"; break;
    // ...
    default: cout << "Invalid";
}
```

REMEMBER

Without 'break', execution falls through to the next case. This is sometimes intentional (e.g., multiple cases share code) but usually a bug.

Ternary Operator — one-line if-else

```
// ternary
// result = (condition) ? value_if_true : value_if_false;
string msg = (age >= 18) ? "Adult" : "Minor";
int max = (a > b) ? a : b;
```

Nested if — conditions inside conditions

```
// nested if
if (num > 0) {
    if (num % 2 == 0) cout << "Positive Even";
    else cout << "Positive Odd";
} else {
    cout << "Non-positive";
}
```

Relational & Logical Operators (used in conditions)

Operator / Syntax	Example	Description
<code>==, !=</code>	<code>a == b, a != b</code>	Equal to, Not equal to
<code>>, >=</code>	<code>a > b, a >= b</code>	Greater than, Greater or equal
<code><, <=</code>	<code>a < b, a <= b</code>	Less than, Less or equal
<code>&&</code>	<code>a>0 && b>0</code>	Logical AND — both must be true
<code> </code>	<code>a>0 b>0</code>	Logical OR — at least one true
<code>!</code>	<code>!(a == b)</code>	Logical NOT — inverts truth



Problem Set — Conditional Statements

#	Level	Conditional Statement Problems
1	Basic	Read a number and check if it is positive, negative, or zero.
2	Basic	Read a year and check whether it is a leap year (divisible by 4, but not 100, unless also 400).
3	Basic	Read two numbers and print the larger one using a ternary operator.
4	Basic	Read a character and check if it is a vowel or consonant using if-else.
5	Basic	Read a single digit (1-7) and print the corresponding day name using a switch statement.
6	Basic	Read three numbers and print the greatest using nested if-else.

7	Basic	Read marks (0-100) and print the letter grade using an if-else-if ladder.
8	Intermediate	Build a BMI calculator: read weight (kg) and height (m), compute BMI, and print the category (Underweight, Normal, Overweight, Obese).
9	Intermediate	Read a 3-digit number. Check if it is an Armstrong number (sum of cubes of digits equals the number).
10	Intermediate	A shop offers 10% discount if purchase > 1000, 20% if > 5000, and 30% if > 10000. Read amount and compute final bill.
11	Intermediate	Read month number and year, then print the number of days in that month (handle February for leap years).
12	Intermediate	Build a simple calculator using switch: read two numbers and an operator (+, -, *, /, %) and display the result.
13	Advanced	A company calculates tax as: 0% up to 2.5L, 5% from 2.5L-5L, 20% from 5L-10L, 30% above 10L. Read income and print tax breakdown.
14	Advanced	Implement a rock-paper-scissors game: read choices for two players and determine the winner using nested conditions.
15	Advanced	A student's final grade depends on: 40% internal, 60% external. Both must be >= 40 to pass. If total >= 90 it's Distinction, >= 75 First Class, >= 60 Second Class. Read marks and determine final result.

C H A P T E R 3

Logical Loops

for, while, do-while, break, continue, nested loops

Cheat Sheet — Logical Loops

```
// Three Loop Types

// 1. for loop — when you know the count
for (int i = 0; i < n; i++) { /* body */ }

// 2. while loop — when you know the condition
while (condition) { /* body */ }

// 3. do-while loop — execute at least once, then check
do { /* body */ } while (condition);
```

break vs continue

Keyword	Effect — used inside a loop body
break	Immediately exits the innermost loop (or switch).
continue	Skips the rest of the current iteration; moves to next cycle.

Nested Loops — loop inside a loop

```
// Nested for loop (print pattern)
for (int i = 1; i <= 5; i++) {      // outer: rows
    for (int j = 1; j <= i; j++) { // inner: columns
        cout << "* ";
    }
    cout << endl;
}
// Output: * / * * / * * * / ...
```

TIP Rule of thumb: use for when count is known, while when condition is unknown, do-while when body must run at least once (e.g., menu-driven programs).

Common Loop Patterns

Pattern	Typical Code
Sum / Average	sum += arr[i]; avg = sum / n;
Factorial	for(i=1;i<=n;i++) fact *= i;
Reverse digits	while(n) { d=n%10; rev=rev*10+d; n/=10; }

Prime check	for(i=2;i*i<=n;i++) if(n%i==0) { notPrime=true; break; }
Fibonacci	a=0,b=1; for(i=0;i<n;i++) { cout<<a; c=a+b; a=b; b=c; }
Count digits	while(n>0) { count++; n/=10; }

Problem Set — Logical Loops

#	Level	Logical Loop Problems
1	Basic	Print all even numbers from 2 to 100 using a for loop.
2	Basic	Calculate the factorial of a number n entered by the user.
3	Basic	Print the Fibonacci series up to n terms.
4	Basic	Read a number and print its multiplication table up to 10.
5	Basic	Read numbers until the user enters 0, then print the sum of all entered numbers.
6	Basic	Print a right-angled triangle pattern of stars with n rows (user-defined).
7	Basic	Reverse a given integer (e.g., 1234 becomes 4321) using a while loop.
8	Intermediate	Check whether a number is prime. Then print all prime numbers between 1 and 100.
9	Intermediate	Print a diamond pattern of stars for a given n (odd number of rows).
10	Intermediate	Read an integer and check whether it is a palindrome (reads same forward and backward).
11	Intermediate	Compute the sum of the series: $1 - 1/2 + 1/3 - 1/4 + \dots$ up to n terms.
12	Intermediate	Print Pascal's triangle for n rows using nested loops.
13	Advanced	Find the GCD and LCM of two numbers using loops (not recursion).
14	Advanced	Print all Armstrong numbers between 1 and 9999 (digits can be 1, 2, 3, or 4 digits).
15	Advanced	Simulate a number guessing game: computer picks a random number 1-100; user guesses; program says Higher/Lower; count attempts and print result on correct guess.

C H A P T E R 4

String Manipulation

std::string, char arrays, string functions

Cheat Sheet — String Manipulation

Declaring & Initializing Strings

```
// String basics
#include <string>
string s1 = "Hello";
string s2(5, 'A');      // "AAAAA"
string s3 = s1 + " World"; // concatenation
getline(cin, s1);      // read with spaces
```

Key std::string Methods

Operator / Syntax	Example	Description
<code>s.length() / size()</code>	<code>s.length()</code>	Number of characters
<code>s.at(i) / s[i]</code>	<code>s[0], s.at(2)</code>	Character at index (at() checks bounds)
<code>s.substr(pos, len)</code>	<code>s.substr(2, 4)</code>	Substring from pos of length len
<code>s.find(t)</code>	<code>s.find("lo")</code>	Index of first occurrence (string::npos if not found)
<code>s.replace(p, l, t)</code>	<code>s.replace(1, 3, "XY")</code>	Replace len chars at pos with t
<code>s.insert(pos, t)</code>	<code>s.insert(2, "AB")</code>	Insert string t at position
<code>s.erase(pos, len)</code>	<code>s.erase(1, 3)</code>	Remove len characters at pos
<code>s.compare(t)</code>	<code>s.compare(t) == 0</code>	Lexicographic comparison (0 = equal)
<code>s.empty()</code>	<code>if (s.empty())</code>	Returns true if string is empty
<code>s.clear()</code>	<code>s.clear()</code>	Empties the string
<code>s.push_back(c)</code>	<code>s.push_back('!')</code>	Append single character
<code>stoi/stod/stol</code>	<code>stoi("42")</code>	String to int/double/long
<code>to_string(n)</code>	<code>to_string(3.14)</code>	Number to string

Character Functions (#include <cctype>)

Function	Returns true when...
<code>isalpha(c)</code>	c is a letter (a-z or A-Z)

<code>isdigit(c)</code>	c is a digit (0-9)
<code>isspace(c)</code>	c is whitespace (space, tab, newline)
<code>isupper(c)</code>	c is uppercase letter
<code>islower(c)</code>	c is lowercase letter
<code>toupper(c)</code>	Returns uppercase version of c
<code>tolower(c)</code>	Returns lowercase version of c



Problem Set — String Manipulation

#	Level	String Manipulation Problems
1	Basic	Read a string and print its length, first character, and last character.
2	Basic	Check whether a string is a palindrome (e.g., 'madam' reads same forward and backward).
3	Basic	Count the number of vowels, consonants, digits, and spaces in a given string.
4	Basic	Convert all lowercase letters in a string to uppercase and vice versa (toggle case).
5	Basic	Reverse a string without using the built-in reverse() function.
6	Basic	Check if a given substring exists in a string, and print its starting position.
7	Basic	Read two strings and concatenate them without using the + operator (use append or insert).
8	Intermediate	Count the frequency of each word in a given sentence (use stringstream to split words).
9	Intermediate	Remove all duplicate characters from a string, keeping only the first occurrence.
10	Intermediate	Given a string, check whether it is an anagram of another string (e.g., 'listen' and 'silent').
11	Intermediate	Implement Caesar cipher: shift each letter in the input string by k positions.
12	Intermediate	Replace every occurrence of a word in a sentence with another word (without using replace()).
13	Advanced	Find the longest palindromic substring in a given string.
14	Advanced	Validate an email address: it must have exactly one @, a domain with a dot, no spaces, and valid characters.
15	Advanced	Compress a string using Run Length Encoding (e.g., 'aaabbc' becomes 'a3b2c1'). Decompress it back.

C H A P T E R 5

Array Functions

1D / 2D arrays, searching, sorting, pointers

Cheat Sheet — Array Functions

```
// Array Declaration & Access
int arr[5] = {10, 20, 30, 40, 50}; // 1D array
int mat[3][3] = {{1,2,3},{4,5,6},{7,8,9}}; // 2D array

// Access: arr[index] (0-based)
arr[0] = 100; // set first element
int x = arr[4]; // get last element
int rows = 3, cols = 3; // store dimensions separately
```

Common Array Operations

Operator / Syntax	Example	Description
Traverse	<code>for(int i=0;i<n;i++) cout<<arr[i];</code>	Visit each element
Find Max/Min	<code>if(arr[i]>max) max=arr[i];</code>	Track max as you traverse
Sum/Average	<code>sum+=arr[i]; avg=sum/n;</code>	Accumulate then divide
Linear Search	<code>if(arr[i]==key) { found=true; }</code>	$O(n)$ — check each element
Binary Search	<code>mid=(lo+hi)/2;</code>	$O(\log n)$ — sorted array only
Bubble Sort	<code>if(a[j]>a[j+1]) swap(a[j],a[j+1]);</code>	$O(n^2)$ — simple comparison sort
Selection Sort	<code>minIdx=i; swap(a[i],a[minIdx]);</code>	$O(n^2)$ — find min, swap to front
Insertion Sort	<code>while(j>=0 && a[j]>key) a[j+1]=a[j]--;</code>	$O(n^2)$ — build sorted portion
2D Traverse	<code>for(i) for(j) cout<<mat[i][j];</code>	Row-major traversal
Array + Pointer	<code>int* p = arr; *(p+i)</code>	Pointer arithmetic on arrays

std::sort and std::algorithm (#include <algorithm>)

Function	Usage
<code>sort(a, a+n)</code>	Sort array ascending ($O(n \log n)$)

<code>sort(a, a+n, greater<int>())</code>	Sort array descending
<code>reverse(a, a+n)</code>	Reverse the array in-place
<code>*max_element(a, a+n)</code>	Pointer to maximum element
<code>*min_element(a, a+n)</code>	Pointer to minimum element
<code>binary_search(a, a+n, v)</code>	Returns true if v found (sorted array)
<code>find(a, a+n, v)</code>	Returns iterator to first occurrence of v
<code>fill(a, a+n, v)</code>	Fill all elements with value v

Problem Set — Array Functions

#	Level	Array Function Problems
1	Basic	Read n integers into an array and print them in reverse order.
2	Basic	Find the maximum, minimum, and average of elements in an array.
3	Basic	Count how many times a given element appears in an array (frequency count).
4	Basic	Merge two sorted arrays into a single sorted array without using sort().
5	Basic	Read a 3x3 matrix and compute its transpose.
6	Basic	Implement linear search: return the index of a key element, or -1 if not found.
7	Basic	Implement bubble sort and count the number of swaps made during sorting.
8	Intermediate	Read a 2D matrix and print: row sums, column sums, and diagonal sums.
9	Intermediate	Rotate an array of n elements by k positions to the left (without using extra array).
10	Intermediate	Find all pairs in an array whose sum equals a given target value k.
11	Intermediate	Implement binary search on a sorted array. Count comparisons made.
12	Intermediate	Remove all duplicates from an unsorted array and print unique elements.
13	Advanced	Multiply two 3x3 matrices and print the result. Verify one row * column manually.
14	Advanced	Find the second largest and second smallest elements in an array in a single traversal.
15	Advanced	Given an array of 0s, 1s, and 2s, sort it in O(n) using the Dutch National Flag algorithm.

C H A P T E R 6

Object-Oriented Programming

Classes, inheritance, polymorphism, encapsulation



Cheat Sheet — OOP

Four Pillars of OOP

Pillar	Meaning in C++
Encapsulation	Bundle data + methods; use private/protected/public to restrict access.
Abstraction	Hide implementation; expose only essential interface (pure virtual, headers).
Inheritance	Derive a new class from existing: <code>class Child : public Parent { }</code> ;
Polymorphism	One interface, many forms: function overloading, operator overloading, virtual functions.

Class Structure

```
// Class anatomy
class BankAccount {
private:                // accessible only inside class
    string owner;
    double balance;

public:                 // accessible everywhere
    BankAccount(string n, double b) : owner(n), balance(b) {}

    void deposit(double amt) { balance += amt; }
    void withdraw(double amt) { if(amt<=balance) balance-=amt; }
    double getBalance() const { return balance; }

    ~BankAccount() { }    // destructor
};
```

Inheritance Types

Type	Syntax & What changes
Public	<code>class B : public A { }</code> ; — public stays public, protected stays protected.
Protected	<code>class B : protected A { }</code> ; — public members become protected.
Private	<code>class B : private A { }</code> ; — all inherited members become private.
Multiple	<code>class C : public A, public B { }</code> ; — inherits from two parents.

Virtual Functions & Polymorphism

```
// runtime polymorphism
class Shape {
public:
    virtual double area() = 0;    // pure virtual => abstract class
    virtual void display() { cout << "Shape: " << area(); }
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() override { return 3.14159 * r * r; }
};

Shape* s = new Circle(5);
s->display();                // calls Circle's area() at runtime
```

NOTE Constructors are called base-first, destructors derived-first. A destructor must be virtual in a base class if you delete derived objects through base pointers.



Problem Set — OOP

#	Level	OOP Problems
1	Basic	Create a class Student with name, rollno, and marks. Add methods to set and display data.
2	Basic	Create a class Rectangle with length and width. Add methods to compute area and perimeter.
3	Basic	Implement a class Counter with increment(), decrement(), and getCount() methods.
4	Basic	Create a class Circle. Overload the + operator to add two circles (sum of radii).
5	Basic	Demonstrate constructor overloading: create a Box class with default, parameterized, and copy constructors.
6	Basic	Create a class with a static variable to count the number of objects created.
7	Basic	Create a base class Animal with a virtual speak() method. Derive Dog, Cat, and Cow and override speak() in each.
8	Intermediate	Design a class BankAccount with deposit, withdraw, and transfer methods. Handle insufficient balance with an exception.
9	Intermediate	Implement a Shape hierarchy: Shape -> (Circle, Triangle, Rectangle). Use polymorphism to compute and display area of any shape via a base pointer.
10	Intermediate	Create a Stack class using an array with push(), pop(), peek(), and isEmpty() methods.

11	Intermediate	Implement a Date class with day, month, year. Overload == and < operators. Add a method to compute days between two dates.
12	Intermediate	Build a Library system: Book class + Library class that manages a collection. Add, remove, and search books.
13	Advanced	Implement a generic Matrix class that supports addition, subtraction, multiplication, and transposition using operator overloading.
14	Advanced	Design an Employee hierarchy: Employee -> (Manager, Developer, HR). Each has a different salary calculation. Use virtual functions and demonstrate runtime polymorphism.
15	Advanced	Build a mini Smart Pointer class (similar to unique_ptr) that auto-deletes memory in destructor and prevents copy but allows move.

C H A P T E R 7

Standard Template Library (STL)

Containers, iterators, algorithms, functors



Cheat Sheet — STL

Sequence Containers

Operator / Syntax	Example	Description
<code>vector<T></code>	<code>v.push_back(x); v[i]; v.size();</code>	Dynamic array. Random access $O(1)$.
<code>list<T></code>	<code>l.push_front(x); l.push_back(x);</code>	Doubly linked list. Fast insert/delete.
<code>deque<T></code>	<code>d.push_front(x); d.push_back(x);</code>	Double-ended queue. Fast both ends.
<code>array<T,N></code>	<code>array<int,5> a = {1,2,3,4,5};</code>	Fixed-size array (C++11). Stack allocated.

Associative Containers

Operator / Syntax	Example	Description
<code>set<T></code>	<code>s.insert(x); s.count(x);</code>	Unique sorted elements. $O(\log n)$ ops.
<code>multiset<T></code>	<code>ms.insert(x);</code>	Like set but allows duplicates.
<code>map<K,V></code>	<code>m[key]=val; m.find(key);</code>	Key-value pairs, sorted by key.
<code>unordered_map<K,V></code>	<code>um[key]=val;</code>	Hash map. Average $O(1)$ lookup.
<code>unordered_set<T></code>	<code>us.insert(x); us.count(x);</code>	Hash set. Average $O(1)$ ops.

Container Adaptors

Operator / Syntax	Example	Description
<code>stack<T></code>	<code>s.push(x); s.top(); s.pop();</code>	LIFO — top access only.
<code>queue<T></code>	<code>q.push(x); q.front(); q.pop();</code>	FIFO — front access only.
<code>priority_queue<T></code>	<code>pq.push(x); pq.top(); pq.pop();</code>	Max-heap by default.

Common STL Algorithms (#include <algorithm>)

Algorithm	Usage
<code>sort(begin, end)</code>	Sort range ascending
<code>binary_search(begin, end, v)</code>	True if v found in sorted range
<code>find(begin, end, v)</code>	Iterator to first occurrence
<code>count(begin, end, v)</code>	Count occurrences of v
<code>accumulate(begin, end, init)</code>	Sum of range (requires <numeric>)
<code>max_element / min_element</code>	Iterator to max/min
<code>unique(begin, end)</code>	Remove consecutive duplicates
<code>next_permutation(begin, end)</code>	Advance to next lexicographic permutation
<code>lower_bound / upper_bound</code>	Binary search bounds on sorted range



Problem Set — STL

#	Level	STL Problems
1	Basic	Use a vector to store n integers, then print them in sorted and reverse-sorted order.
2	Basic	Use a stack to check whether a string of brackets {}, [], () is balanced.
3	Basic	Use a queue to simulate a print spooler: add 5 jobs, then process them in order.
4	Basic	Use a set to find all unique elements in an array.
5	Basic	Use a map to count the frequency of each word in a sentence.
6	Basic	Use priority_queue to find the 3 largest elements in an array.
7	Basic	Use STL sort with a custom comparator to sort strings by their length.
8	Intermediate	Use a map<string, vector<string>> to implement a simple phonebook: store, search, and display contacts.
9	Intermediate	Use a multimap to group students by their grade (A, B, C) and list them per grade.
10	Intermediate	Implement a frequency chart using unordered_map and print the top 3 most frequent elements.
11	Intermediate	Given two sets, compute and print their union, intersection, and difference.
12	Intermediate	Use next_permutation to generate all permutations of a string.
13	Advanced	Implement a simple LRU Cache using a combination of list and unordered_map.
14	Advanced	Use a priority_queue to simulate the Huffman encoding algorithm and print the frequency table.
15	Advanced	Given a log file (vector of strings), use STL containers and algorithms to find the top 5 most frequent IP addresses.

C H A P T E R 8

Recursion, Templates & Lambdas (RTL)

Recursive thinking, generic programming, modern C++



Cheat Sheet — RTL

Recursion — a function that calls itself

```
// Recursion anatomy
int factorial(int n) {
    if (n <= 1) return 1;           // BASE CASE — must always exist!
    return n * factorial(n - 1);    // RECURSIVE CASE
}

// Think in 3 steps:
// 1. What is the base case? (smallest subproblem with known answer)
// 2. What is the recursive relation? (how to reduce problem size)
// 3. Trust: assume recursive call works and build on it.
```

Function Templates — generic functions

```
// Templates
template <typename T>
T maxVal(T a, T b) { return (a > b) ? a : b; }

// Works for any type
maxVal(3, 5);           // int
maxVal(3.14, 2.71);     // double
maxVal('a', 'z');       // char

// Class Template
template <typename T>
class Pair { T first, second; };
```

Lambda Expressions (C++11) — anonymous inline functions

```
// Lambda
// Syntax: [capture](params) -> return_type { body }
auto square = [](int x) { return x * x; };
cout << square(5);    // 25

// Capture by value [=] or by reference [&]
int factor = 3;
auto multiply = [factor](int x) { return x * factor; };
```

```
// Use with STL algorithms
sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
for_each(v.begin(), v.end(), [](int x) { cout << x << ' '; });
```

REMEMBER

Every recursive function must have a base case that is reached eventually. Missing or wrong base cases cause infinite recursion and a stack overflow.



Problem Set — Recursion, Templates & Lambdas

#	Level	RTL Problems
1	Basic	Write a recursive function to compute the nth Fibonacci number.
2	Basic	Write a recursive function to reverse a string.
3	Basic	Write a recursive function to compute the sum of digits of an integer.
4	Basic	Write a function template swap<T> that swaps two values of any type.
5	Basic	Write a lambda expression that checks if a number is even, and apply it to a vector using count_if.
6	Basic	Write a recursive function to check if a string is a palindrome.
7	Basic	Write a function template to find the maximum of three values of any numeric type.
8	Intermediate	Write a recursive function to implement binary search on a sorted array.
9	Intermediate	Use recursion to print all subsets (power set) of a given set {1, 2, 3, n}.
10	Intermediate	Write a class template Stack<T> that uses a vector internally and supports push, pop, and top.
11	Intermediate	Sort a vector of Student objects using a lambda, first by grade descending, then by name ascending.
12	Intermediate	Use recursion to solve the Tower of Hanoi problem for n disks and print each move.
13	Advanced	Write a variadic template function that takes any number of arguments and prints them all (similar to printf but type-safe).
14	Advanced	Use memoization (cache in a map) to make recursive Fibonacci O(n) instead of O(2^n). Compare times for n=40.
15	Advanced	Implement a recursive merge sort using templates so it works for both int and string vectors.

C H A P T E R 9

File Handling

ifstream, ofstream, fstream, binary I/O, seekg/seekp



Cheat Sheet — File Handling

Operator / Syntax	Example	Description
<code>ifstream</code>	<code>ifstream fin("a.txt");</code>	Input file stream (read)
<code>ofstream</code>	<code>ofstream fout("a.txt");</code>	Output file stream (write)
<code>fstream</code>	<code>fstream f("a.txt", ios::in ios::out);</code>	Read + write
<code>is_open()</code>	<code>if (!fin.is_open())</code>	Check if file opened successfully
<code>getline()</code>	<code>getline(fin, line);</code>	Read entire line
<code>>></code>	<code>fin >> word;</code>	Read word/value from file
<code><<</code>	<code>fout << data << endl;</code>	Write data to file
<code>close()</code>	<code>fin.close();</code>	Close file (auto on destruction)
<code>eof()</code>	<code>while(!fin.eof())</code>	True when end-of-file reached
<code>seekg(pos, origin)</code>	<code>fin.seekg(0, ios::beg);</code>	Move read pointer (beg/cur/end)
<code>seekp(pos, origin)</code>	<code>fout.seekp(0, ios::end);</code>	Move write pointer
<code>tellg()</code>	<code>streampos p = fin.tellg();</code>	Get current read position
<code>ios::app</code>	<code>ofstream f("log.txt", ios::app);</code>	Append without erasing
<code>ios::binary</code>	<code>ios::in ios::binary</code>	Binary mode — raw bytes
<code>read/write</code>	<code>f.read((char*)&obj, sizeof(obj));</code>	Binary struct read/write



Problem Set — File Handling

#	Level	File Handling Problems
1	Basic	Write a program to create a file 'hello.txt', write 'Hello, World!' to it, then read and print its contents.
2	Basic	Count the total number of lines, words, and characters in a given text file.
3	Basic	Copy the contents of one text file to another file.
4	Basic	Append a new line to an existing log file with the current timestamp (use time.h).

5	Basic	Read a file containing 10 integers (one per line), find and print the max, min, and average.
6	Basic	Write student records (name, rollno, marks) to a text file and read them back in a formatted table.
7	Intermediate	Search for a specific word in a text file. Print the line number and line content each time it is found.
8	Intermediate	Write student objects to a binary file using write(). Read them back and display using read().
9	Intermediate	Implement a simple phonebook stored in a file: add, search, and delete entries (update file in-place).
10	Intermediate	Use seekg() to read the Nth record from a binary file without reading all preceding records.
11	Intermediate	Merge two sorted text files (each containing integers one per line) into a single sorted output file.
12	Advanced	Build a mini CSV parser: read a CSV file, store rows in a vector<vector<string>>, and print a formatted table.
13	Advanced	Implement a text file encryptor/decryptor using XOR with a secret key. Encrypt a file and verify by decrypting it back.
14	Advanced	Create a student database system using binary files: support add, search by rollno, update marks, and delete student — all persistent across runs.

C H A P T E R 1 0

Exception Handling

try, catch, throw, custom exceptions, exception hierarchy

Cheat Sheet — Exception Handling

```
// Basic try-catch-throw
try {
    if (b == 0) throw runtime_error("Division by zero!");
    cout << a / b;
}
catch (const runtime_error& e) {
    cerr << "Error: " << e.what() << endl;
}
catch (...) {           // catch-all - handles any exception
    cerr << "Unknown error occurred." << endl;
}
```

Standard Exception Classes (#include <stdexcept>)

Exception Class	When to throw it
<code>std::exception</code>	Base class for all standard exceptions. Override what().
<code>std::runtime_error</code>	Errors detectable only at runtime (bad input, file missing).
<code>std::logic_error</code>	Logic errors in program (invalid argument, out of range).
<code>std::invalid_argument</code>	Function received invalid argument.
<code>std::out_of_range</code>	Index or value outside valid range.
<code>std::overflow_error</code>	Arithmetic overflow.
<code>std::bad_alloc</code>	new fails to allocate memory.

Custom Exception Class

```
// Custom exception
class InsufficientFundsException : public exception {
    double amount;
public:
    InsufficientFundsException(double a) : amount(a) {}
    const char* what() const noexcept override {
        return "Insufficient funds!";
    }
    double getAmount() const { return amount; }
};
```

```
// Throw it
if (balance < amount)
    throw InsufficientFundsException(amount);
```

Exception Safety Guarantees

Guarantee	What it means
No-throw	Function never throws (noexcept). Destructors and move ops must be no-throw.
Strong	If exception thrown, program state is unchanged (rollback).
Basic	No resource leaks; objects still in valid state, but state may change.
None	No guarantee — avoid this.
TIP	Catch by const reference (const ExcType& e) — never by value (copies) or by pointer (memory issues). Order catches from most-specific to most-general, since C++ matches the first applicable handler.



Problem Set — Exception Handling

#	Level	Exception Handling Problems
1	Basic	Write a program that divides two numbers. Use try-catch to handle division by zero.
2	Basic	Access an element of a vector using .at(). Catch std::out_of_range if the index is invalid.
3	Basic	Write a function that takes a positive integer only. Throw std::invalid_argument if a negative is passed.
4	Basic	Demonstrate the catch-all handler (...): throw an int, a string, and a double and catch each differently.
5	Basic	Open a file. If it does not exist, throw and catch a runtime_error with a meaningful message.
6	Basic	Write a safe integer square root function that throws std::domain_error for negative inputs.
7	Basic	Demonstrate stack unwinding: throw an exception inside a function 3 levels deep and catch it in main().
8	Intermediate	Create a custom exception class AgeException. Throw it when age < 0 or age > 150. Catch and display the invalid value.
9	Intermediate	Build a safe BankAccount class: throw InsufficientFundsException on overdraft and NegativeAmountException on negative deposit.
10	Intermediate	Write a function that converts a string to integer. Throw exceptions for: empty string, non-numeric characters, and overflow.
11	Intermediate	Demonstrate rethrowing: catch an exception, log it, then rethrow it for the caller to handle.
12	Intermediate	Implement a safe matrix class: throw out_of_range on invalid row/column access, logic_error on incompatible dimensions for multiply.

13	Advanced	Design an exception hierarchy for a Hospital system: HospitalException -> (PatientNotFoundException, BedNotAvailableException, MedicationException). Demonstrate catching at different levels.
14	Advanced	Build a transaction system: if any step (debit, credit, log) throws an exception, roll back all previous steps using RAI and noexcept guarantees.
15	Advanced	Write a multi-threaded safe exception handler: two threads each perform risky operations; capture exceptions from both threads in the main thread using std::exception_ptr and rethrow.

Logic Building — Master Tips

After completing all 10 chapters, these principles will guide you from good programmer to great one:

Principle	What to practice
Draw before you code	Sketch the flow, data structure, or class diagram on paper. Code follows design.
Name things clearly	totalStudentMarks not tsm. Code is read far more than it is written.
Handle every edge case	Empty input, zero, negative numbers, maximum values — test all boundaries.
One function = one job	If you cannot describe a function in one sentence, split it.
Measure, don't guess	Use a loop counter or timer to verify your $O(n)$ assumption.
Read error messages	Compiler errors are your friend — read the first error, fix it, recompile.
Test incrementally	Add one feature at a time. Compile and test after every addition.
Learn from solutions	After solving a problem, look at elegant solutions and ask 'what did I miss?'

Dr. Amar Panchal is on a mission to give Digital India a boost by training the brightest minds of India to achieve their dreams of building successful tech careers.

He uses one of his major tools,
 **Career Credentials**
 empowered by his teaching skill,
 one-on-one mentorship, batches,
 online & offline workshops, and YouTube
 to make a difference.

Connect with him,
 scan the QR to know more
 about him & use free resources
 provided by him.



Scan to Connect!

www.careercredentials.in

Best of luck for the future! - Dr. Amar Panchal & Career Credentials